

Devoir surveillé

Durée 2h
(Corrections)

Nom :
Prénom :
Num. inscription :

Aucun document, outil de calcul ou de communication autorisé.

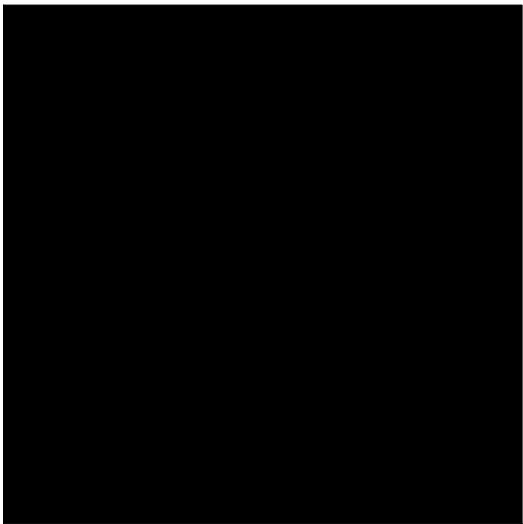
Exercice 1 (QCM - 5 pts)

Vous obtiendrez 0,5 point aux questions si vous cochez exactement toutes les bonnes réponses. Une mauvaise réponse cochée entraîne des points négatifs.

- Avec un *i-node*, sous *ext2*...
 - ☐ ...les blocs de donnée d'un fichier sont stockés de manière contigüe sur le disque
 - ☒ ...les blocs de donnée d'un fichier peuvent être stockés à différents endroits sur le disque
 - ☒ ...la taille maximale d'un fichier dépend de la taille des blocs du disque
 - ☒ ...un bloc de données ne peut pas contenir les données de plusieurs fichiers
- Quelle(s) affirmation(s) est/sont juste(s) ?
 - ☒ La taille d'un fichier peut être inférieure à la taille occupée sur le disque
 - ☐ La taille d'un fichier est identique à la taille occupée sur le disque
 - ☒ La taille d'un fichier peut être supérieure à la taille occupée sur le disque
 - ☒ La taille occupée sur le disque dépend de la taille des blocs
- Concernant les structures en C :
 - ☒ Elles sont alignées automatiquement par le compilateur
 - ☒ Leur taille dépend de l'ordre de définition des champs
 - ☐ Il est plus efficace de réaliser des allocations dynamiques pour les utiliser
 - ☐ Pour une question de portabilité, il faut éviter de les utiliser
- Sur une architecture 64 bits, quelle est la taille de la structure suivante en utilisant `gcc` sous Linux ?

```
typedef struct {  
    long a;  
    char b;  
    char c;  
    int d;  
    short e;  
} structure_t;
```

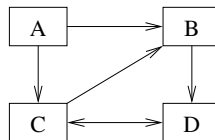
- ☐ 16o ☐ 20o ☒ 24o ☐ 32o
- En considérant la question 4, combien d'octets de bourrage sont ajoutés par le compilateur ?
 - ☐ 0 ☐ 4 ☒ 8 ☐ Une autre réponse
 - En considérant la question 4, quelle est la taille minimale que l'on peut obtenir en réarrangeant les champs de la structure (sans modifier les types) ?
 - ☐ 8 ☐ 12 ☐ 18 ☒ Une autre réponse



7. Les tubes anonymes :

- ☐ Ils ne permettent pas l'écriture simultanée de plusieurs processus
- ☐ L'écriture y est atomique
- ☐ La création d'un tube anonyme entraîne l'ajout d'une entrée dans le système de fichiers
- ☒ La création nécessite deux descripteurs de fichier

8. Combien de sémaphores de *Dijkstra* sont nécessaires au minimum pour résoudre les contraintes du diagramme de précedence suivant (A, B, C et D étant des sections de code de processus quelconques) ?

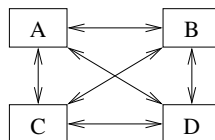


- ☐ 1 ☐ 2 ☒ 3 ☐ 4

9. Soit le diagramme de précedence de la question 8. Combien d'opérations (hors `init`) sont nécessaires ?

- ☐ 4 ☐ 6 ☒ 8 ☐ 10

10. Combien de sémaphores de *Dijkstra* sont nécessaires au minimum pour résoudre les contraintes du diagramme de précedence suivant (A, B, C et D étant des sections de code de processus quelconques) ?



- ☒ 1 ☐ 3 ☐ 6 ☐ 12

Annexes : tailles et alignements

Nous considérons les tailles et les alignements suivants (correspondant à *gcc* sous *Linux*, sur une architecture 64 bits) :

Type	Taille	Aligne.	Type	Taille	Aligne.	Type	Taille	Aligne.
char	1o	1o	short	2o	2o	int	4o	4o
long	8o	8o	float	4o	4o	double	8o	8o
long long	8o	8o	long double	16o	16o	pointer	8o	8o

*Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées. Tout réponse doit être justifiée.*

Exercice 2 (L'hydre de Lerne à n têtes - 7 pts)

L'hydre de Lerne est un animal mythologique constitué de plusieurs têtes : lorsque l'une d'elles est coupée, elle repousse. Par contre, si toutes les têtes sont coupées simultanément, l'hydre meurt (en tout cas, nous utiliserons cette interprétation pour cet exercice).

Nous souhaitons simuler le comportement de l'hydre à l'aide d'un processus et de n processus fils (qui correspondent aux têtes). Pour trancher une tête, nous envoyons un signal `SIGUSR1` à l'un des fils. Le père crée un nouveau fils lorsqu'un fils meurt sauf si tous ses fils sont morts.

1°) Quel mécanisme système permet à un processus de savoir qu'un de ses fils est terminé ? Dans un programme en C, comment un processus peut-il savoir qu'un de ses fils s'arrête ? Que doit-il faire pour libérer les ressources associées au fils ?

Solution : (1 point) Le signal `SIGCHLD` est envoyé au père lorsqu'un fils meurt. Dans le programme, le père peut créer les fils puis attendre à l'aide d'un `wait`. Cet appel sera bloquant jusqu'à la fin d'un fils dont les ressources seront alors libérées.

2°) Un fils se contente d'attendre la réception d'un signal `SIGUSR1` : à l'aide d'une alarme, il affiche régulièrement le message "*Je suis en vie (X) !*" (où X est son PID). Il affiche le message "*Je me suis fait couper (X)*" (où X est son PID) lorsqu'il reçoit le signal fatidique. Donnez le code de la procédure `fils` correspondant à la routine d'un fils. Vous donnerez les fonctions/procédures supplémentaires si nécessaires.

Solution : (2 points) Voici une solution :

```
int stop = 0;
void gestionnaire(int numsig) {
    if(numsig == SIGUSR) {
        alarm(0);
        printf("Je me suis fait couper !\n");
        stop = 1;
    }
    if(numsig == SIGALRM)
        printf("Je suis en vie (%d)\n", getpid());
}
void fils() {
    struct sigaction action;

    action.sa_handler = gestionnaire;
    sigfillset(&action.sa_mask);
    action.sa_flags = 0;
    sigaction(SIGUSR1, &action, NULL);
    sigaction(SIGALRM, &action, NULL);

    while(stop == 0) {
        alarm(5);
        pause();
    }
    exit(EXIT_SUCCESS);
}
```

3°) Le programme principal prend le nombre de fils en argument. Il crée les fils puis se met en attente de la mort de l'un d'eux. Pour le moment, il ne crée pas de nouvelle tête (de fils) et il s'arrête dès qu'il n'en a plus. Donnez le code du `main`.

Solution : (1,5 points) Voici une solution :

```

int main(int argc, char *argv[]) {
    int nbfiles, i, n, restants;

    if(argc != 2) {
        fprintf(stderr, "Pas assez d'arguments\n");
        exit(EXIT_FAILURE);
    }
    nbfiles = atoi(argv[1]);
    for(i = 0; i < nbfiles; i++) {
        if(fork() == 0)
            fils();
    }
    restants = nbfiles;
    while(restants != 0) {
        wait(NULL);
        restants--;
    }

    return EXIT_SUCCESS;
}

```

4°) Que se passe-t-il si c'est le père qui reçoit par inadvertance le signal **SIGUSR1** ?

Solution : (0,5 point) Le père meurt (car c'est le comportement normal à la réception d'un **SIGUSR1**. Les fils continuent leur exécution.

Pour faire repousser les têtes, nous utilisons une alarme. Le père arme une alarme de cinq secondes à la mort de l'un de ses fils. Si l'alarme arrive à expiration, le père crée les fils manquants. S'il perd un autre fils entre temps, il réarme l'alarme. Nous affichons un message "Je vais pouvoir faire pousser mes têtes" à la fin de l'alarme à l'aide d'un gestionnaire.

5°) Que se passe-t-il pour un appel système lorsqu'un processus reçoit un signal ? Comment s'en rendre compte dans le programme ?

Solution : (1 point) L'appel système est interrompu si un gestionnaire est associé au signal (s'il n'est pas ignoré). L'appel échoue et retourne la valeur -1. La variable **errno** prend l'erreur **EINTR**.

6°) Expliquez comment mettre en place la gestion de l'alarme, ainsi que les modifications à apporter à la boucle principale.

Solution : (1 point) Tout d'abord, on place un gestionnaire sur la réception d'une alarme. Il affichera simplement le message spécifié. Dans la boucle principale, si **wait** échoue, c'est que l'alarme s'est déclenchée. Donc, on crée les fils manquants. Sinon, un fils s'est arrêté et il faut réarmer l'alarme de cinq secondes.

```

while(restants != 0) {
    if(wait(NULL) == -1) {
        for(i = restant; i < nbfiles; i++)
            if(fork() == 0)
                fils();
        restants = nbfiles;
    }
    else {
        alarme(5);
        restants--;
    }
}

```

Exercice 3 (Redirection - 3 pts)

Nous souhaitons réaliser un programme que nous appellerons l'*exécuteur*, qui exécute un second programme

que nous appellerons le *programme client*. Le programme client lit au clavier cinq entiers puis cinq chaînes de caractères. L'exécuteur prend comme arguments le nom du programme client, ainsi que le nom d'un fichier qui contient les données à envoyer au programme client via l'entrée standard.

1°) Rappelez comment un programme peut exécuter un second programme, tout en continuant son exécution (le premier programme doit attendre la fin de l'exécution du second programme). Vous préciserez en plus, tous les appels systèmes nécessaires.

Solution : (1 point) L'appel système *execve* permet de remplacer l'exécution en cours par l'exécution d'un programme donné. Aussi, il est nécessaire de créer un fils qui lancera l'appel à *execve*. Nous aurons donc besoin de *fork* pour créer le fils et pour attendre la fin du programme, nous utiliserons *wait*.

2°) Expliquez comment *dup* (ou *dup2*) peut être utilisé pour que le programme client lise les données depuis le fichier au lieu de l'entrée standard. À quel niveau, dans le programme de l'exécuteur, doit-on faire l'appel à *dup* ?

Solution : (1 point) *dup* permet de dupliquer un descripteur de fichier. Si le fils de l'exécuteur ouvre le fichier d'entrée en lecture, puis duplique le descripteur de fichier associé en remplaçant le flux d'entrée (*STDIN_FILENO*), les données seront lues depuis le fichier et lues dans le programme client. En effet, *execve* conserve les descripteurs de fichier ouverts.

Nous souhaitons maintenant rediriger la sortie standard du programme client dans un tube anonyme. L'exécuteur récupère alors les données écrites.

3°) Expliquez ce que l'on doit mettre en place et comment. Vous donnez en plus les appels systèmes nécessaires.

Solution : (1 point) L'exécuteur doit créer le tube anonyme (à l'aide de *pipe*) avant la création du fils et lui transmettre le descripteur de fichier de sortie. Dans le fils, nous faisons un second appel à *dup* pour rediriger la sortie standard dans le descripteur spécifié. Dans le père, il suffit qu'il se mette en attente de lecture sur le tube jusqu'à ce que *read* retourne 0 (plus de donnée).

Exercice 4 (Tubes - 5 pts)

Nous souhaitons faire communiquer deux processus à l'aide de deux tubes nommés.

1°) Quelle(s) est/sont la/les différence(s) entre un tube nommé et un tube anonyme ?

Solution : (1 point) Un tube nommé est associé à une entrée dans le système de fichiers, ce qui permet à deux processus sans lien de parenté (direct ou lointain) de communiquer via le tube. L'ouverture du tube est bloquante. Pour un tube anonyme, il est nécessaire de transmettre les descripteurs de fichier.

Le premier programme crée les deux tubes nommés puis les ouvre. Le second programme se contente de les ouvrir (en considérant que les tubes sont créés et qu'il soit donc exécuté en second). Les noms des tubes sont spécifiés en constante (*NOM_TUBE1* et *NOM_TUBE2*).

2°) Donnez la portion de code du premier programme permettant de créer les tubes nommés et de les ouvrir.

Solution : (1 point)

```
int fd1, fd2;

mkfifo(NOM_TUBE1, S_IRUSR | S_IWUSR);
mkfifo(NOM_TUBE2, S_IRUSR | S_IWUSR);
fd1 = open(NOM_TUBE1, O_RDONLY);
fd2 = open(NOM_TUBE2, O_WRONLY);
```

3°) Donnez la portion de code du second programme permettant d'ouvrir les tubes nommés.

Solution : (0,5 point)

```
int fd1, fd2;

fd1 = open(NOM_TUBE1, O_WRONLY);
fd2 = open(NOM_TUBE2, O_RDONLY);
```

Nous supposons que les programmes s'échangent des messages dont la structure est la suivante :

```
typedef struct {
    pid_t PID;
    char *msg;
} message_t;
```

4°) Donnez la portion de code permettant d'initialiser puis d'écrire le contenu d'une telle structure dans un tube depuis le premier programme. Le champ `PID` contient le PID du processus courant et le champ `msg` contient une chaîne de caractères lue au clavier.

Solution : (2 points)

```
message_t msg;
size_t taille;

msg.PID = getpid();
msg.msg = (char*)malloc(sizeof(char) * 256);
scanf("%255s", msg.msg);
taille = strlen(msg.msg) + 1;
write(fd1, &msg.PID, sizeof(pid_t));
write(fd1, &taille, sizeof(size_t));
write(fd1, msg.msg, taille);
```

5°) Nous souhaitons que le premier programme supprime les tubes nommés dès qu'il se termine (de manière normale ou non). Expliquez comment faire et donnez les instructions à ajouter.

Solution : (0,5 point) On peut utiliser la fonction `atexit` qui permet d'enregistrer une procédure à appeler. Cette procédure supprime les tubes nommés à l'aide de `unlink`.

```
/* Enregistrement */
atexit(fin);

/* Procédure */
void fin() {
    unlink(NOM_TUBE1);
    unlink(NOM_TUBE2);
}
```